SQL

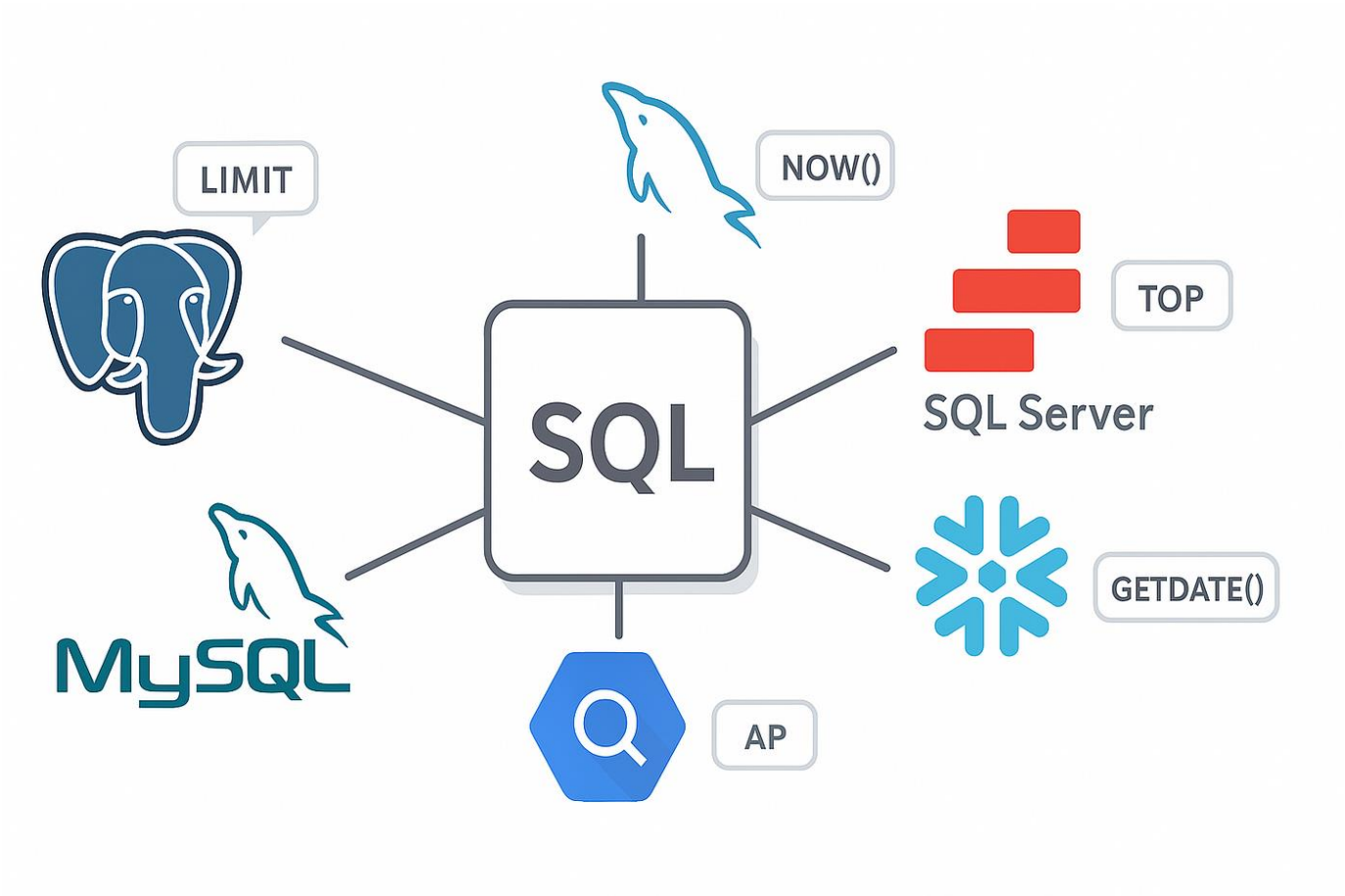
Лекция №3

Роль SQL в ETL

SQL используется для:

1. Трансформации данных
2. Очистки данных
3. Объединения источников
4. Построения агрегированных таблиц
5. Подготовки витрин данных

Много диалектов



SELECT

```sql
SELECT *
  FROM transactions;
```

	transaction_id	user_id	amount	currency	event_time
1	1	1	100	EUR	2024-01-01 10:15:00.000
2	2	1	50	EUR	2024-01-02 11:00:00.000
3	3	2	200	EUR	2024-01-02 12:30:00.000
4	4	3	150	EUR	2024-01-03 09:45:00.000
5	5	3	-20	EUR	2024-01-04 13:20:00.000
6	6	4	300	EUR	2024-01-05 16:10:00.000
7	7	5	120	EUR	2024-01-06 08:05:00.000
8	4	3	150	EUR	2024-01-03 09:45:00.000
9	2	1	50	EUR	2024-01-02 11:00:00.000
10	8	1	70	EUR	2024-01-07 10:00:00.000
11	9	1	30	EUR	2024-01-08 12:00:00.000
12	10	2	90	EUR	2024-01-09 14:00:00.000

```sql
SELECT transaction_id, amount
  FROM transactions;
```

	transaction_id	amount
1	1	100
2	2	50
3	3	200
4	4	150
5	5	-20
6	6	300
7	7	120
8	4	150

SELECT DISTINCT

```
SELECT distinct transaction_id, amount
  FROM transactions;
```

	123 transaction_id	123 amount
1	9	30
2	1	100
3	6	300
4	4	150
5	3	200
6	10	90
7	2	50
8	5	-20
9	7	120
10	8	70

WHERE

```sql
SELECT *
  FROM transactions
 WHERE amount > 0;
```

	transaction_id	user_id	amount	currency	event_time
1	1	1	100	EUR	2024-01-01 10:15:00.000
2	2	1	50	EUR	2024-01-02 11:00:00.000
3	3	2	200	EUR	2024-01-02 12:30:00.000
4	4	3	150	EUR	2024-01-03 09:45:00.000
5	6	4	300	EUR	2024-01-05 16:10:00.000
6	7	5	120	EUR	2024-01-06 08:05:00.000
7	4	3	150	EUR	2024-01-03 09:45:00.000
8	2	1	50	EUR	2024-01-02 11:00:00.000
9	8	1	70	EUR	2024-01-07 10:00:00.000
10	9	1	30	EUR	2024-01-08 12:00:00.000
11	10	2	90	EUR	2024-01-09 14:00:00.000

WHERE p2

```sql
SELECT *
  FROM transactions
 WHERE (amount > 0
        AND user_id = 1)
       OR EVENT_TIME <= date'2024-01-05';
```

	transaction_id	user_id	amount	currency	event_time
1	1	1	100	EUR	2024-01-01 10:15:00.000
2	2	1	50	EUR	2024-01-02 11:00:00.000
3	3	2	200	EUR	2024-01-02 12:30:00.000
4	4	3	150	EUR	2024-01-03 09:45:00.000
5	5	3	-20	EUR	2024-01-04 13:20:00.000

ORDER BY

```sql
SELECT *
  FROM transactions
 ORDER BY event_time DESC;
```

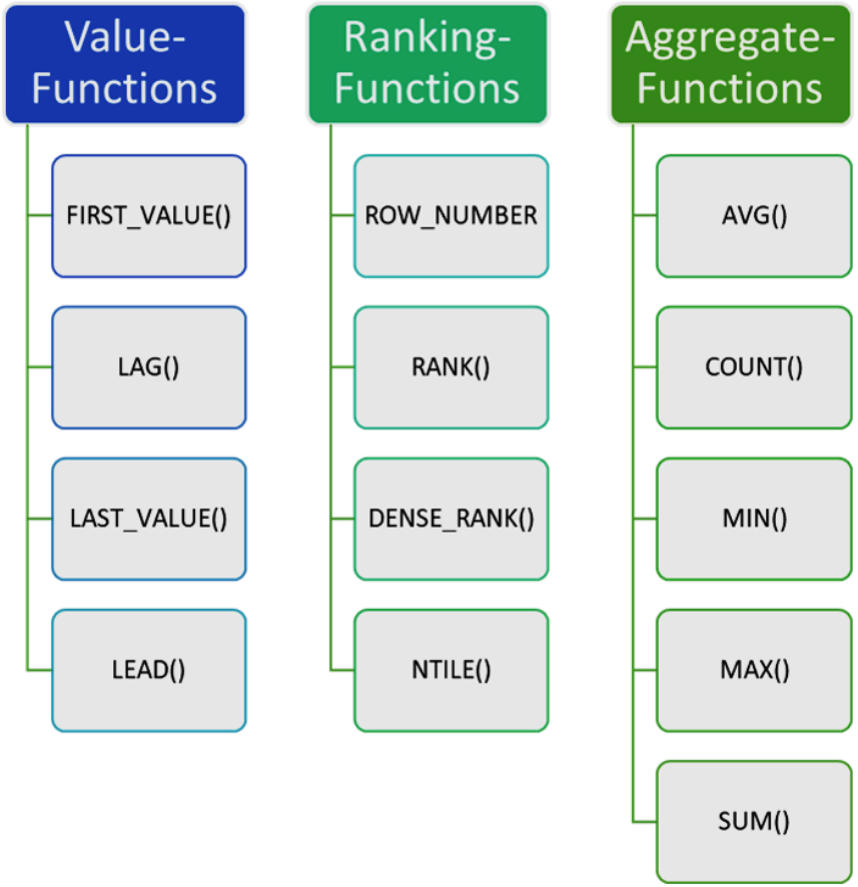
	transaction_id	user_id	amount	currency	event_time
1	10	2	90	EUR	2024-01-09 14:00:00.000
2	9	1	30	EUR	2024-01-08 12:00:00.000
3	8	1	70	EUR	2024-01-07 10:00:00.000
4	7	5	120	EUR	2024-01-06 08:05:00.000
5	6	4	300	EUR	2024-01-05 16:10:00.000
6	5	3	-20	EUR	2024-01-04 13:20:00.000
7	4	3	150	EUR	2024-01-03 09:45:00.000
8	4	3	150	EUR	2024-01-03 09:45:00.000
9	3	2	200	EUR	2024-01-02 12:30:00.000
10	2	1	50	EUR	2024-01-02 11:00:00.000
11	2	1	50	EUR	2024-01-02 11:00:00.000
12	1	1	100	EUR	2024-01-01 10:15:00.000

ORDER BY

```sql
SELECT *
  FROM transactions
 ORDER BY 2;
```

	transaction_id	user_id	amount	currency	event_time
1	8	1	70	EUR	2024-01-07 10:00:00.000
2	2	1	50	EUR	2024-01-02 11:00:00.000
3	2	1	50	EUR	2024-01-02 11:00:00.000
4	1	1	100	EUR	2024-01-01 10:15:00.000
5	9	1	30	EUR	2024-01-08 12:00:00.000
6	10	2	90	EUR	2024-01-09 14:00:00.000
7	3	2	200	EUR	2024-01-02 12:30:00.000

Агрегации




```sql
SELECT user_id, SUM(amount)
  FROM transactions
 GROUP BY user_id;
```

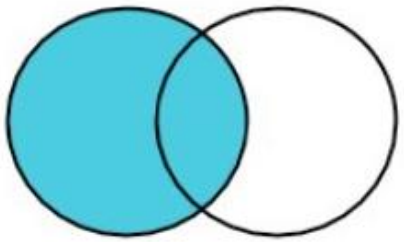
	123 user_id	123 sum
1	3	280
2	5	120
3	4	300
4	2	290
5	1	300

JOIN

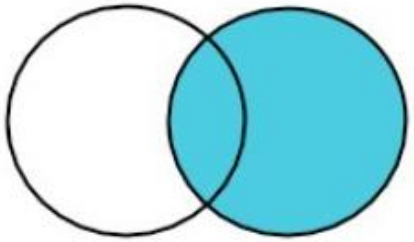
```sql
SELECT t.transaction_id,
       u.name,
       u.country,
       t.amount
  FROM transactions t
       JOIN users u
       ON t.user_id = u.user_id;
```

	123 transaction_id	A-Z name	A-Z country	123 amount
1	1	Alice	DE	100
2	2	Alice	DE	50
3	3	Bob	NL	200
4	4	Charlie	DE	150
5	5	Charlie	DE	-20
6	6	Diana	FR	300
7	7	Eve	NL	120
8	4	Charlie	DE	150
9	2	Alice	DE	50
10	8	Alice	DE	70
11	9	Alice	DE	30
12	10	Bob	NL	90

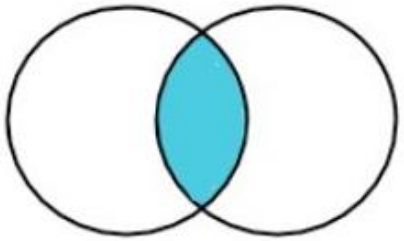
SQL Joins



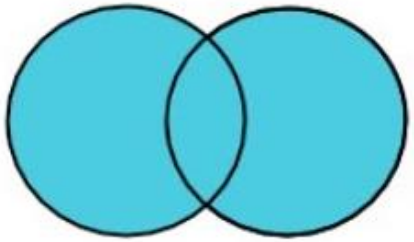
Left Join



Right Join



Inner Join



Full Outer Join

Оконные функции

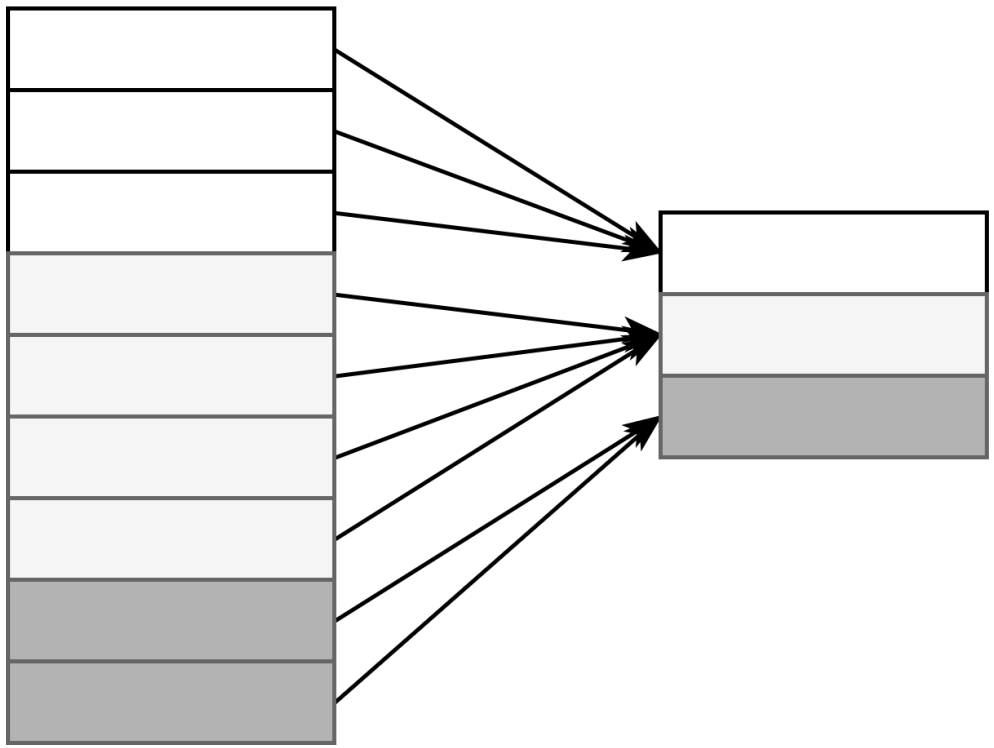
```sql
SELECT user_id,
       amount,
       SUM(amount) over (partition by user_id) AS total_amount
  FROM transactions;
```

	123 user_id	123 amount	123 total_amount
1	1	70	300
2	1	50	300
3	1	50	300
4	1	100	300
5	1	30	300
6	2	90	290
7	2	200	290
8	3	150	280
9	3	-20	280
10	3	150	280
11	4	300	300
12	5	120	120

GROUP BY

Table data

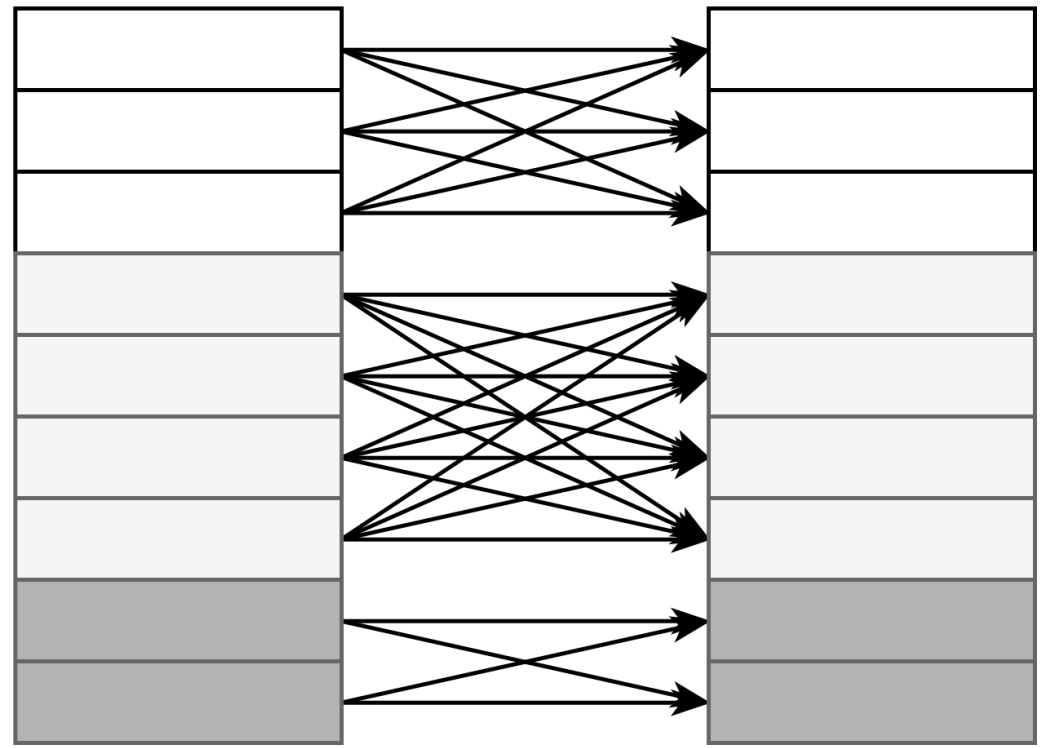
Result set



Window functions

Table data

Result set



CTE

```sql
WITH clean_data AS (
    SELECT *
      FROM transactions
     WHERE amount > 0
)
SELECT user_id,
       SUM(amount)
  FROM clean_data
 GROUP BY user_id;
```

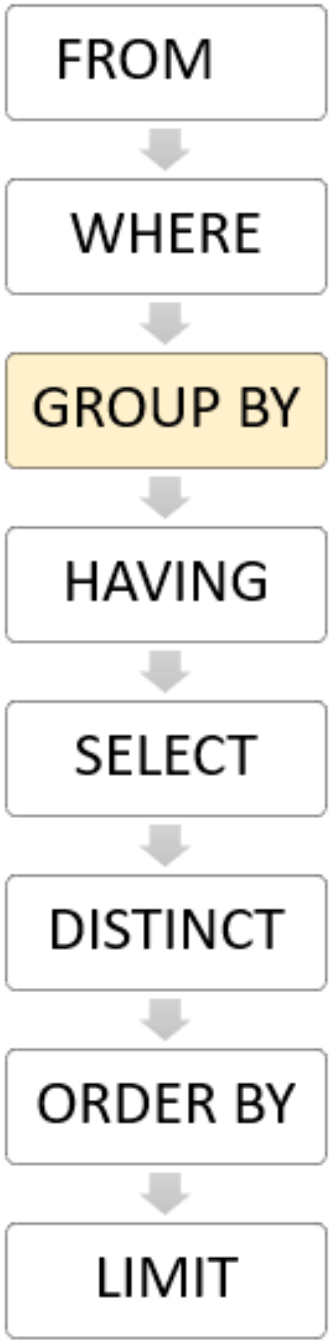
	123 user_id	123 sum
1	3	300
2	5	120
3	4	300
4	2	290
5	1	300

SQL как ETL

```sql
WITH clean_transactions AS (
    SELECT *
      FROM transactions
     WHERE amount > 0
),
joined_data AS (
    SELECT t.*,
           u.country
      FROM clean_transactions t
           JOIN users u
           ON t.user_id = u.user_id
)
SELECT country,
       SUM(amount)
  FROM joined_data
 GROUP BY country;
```

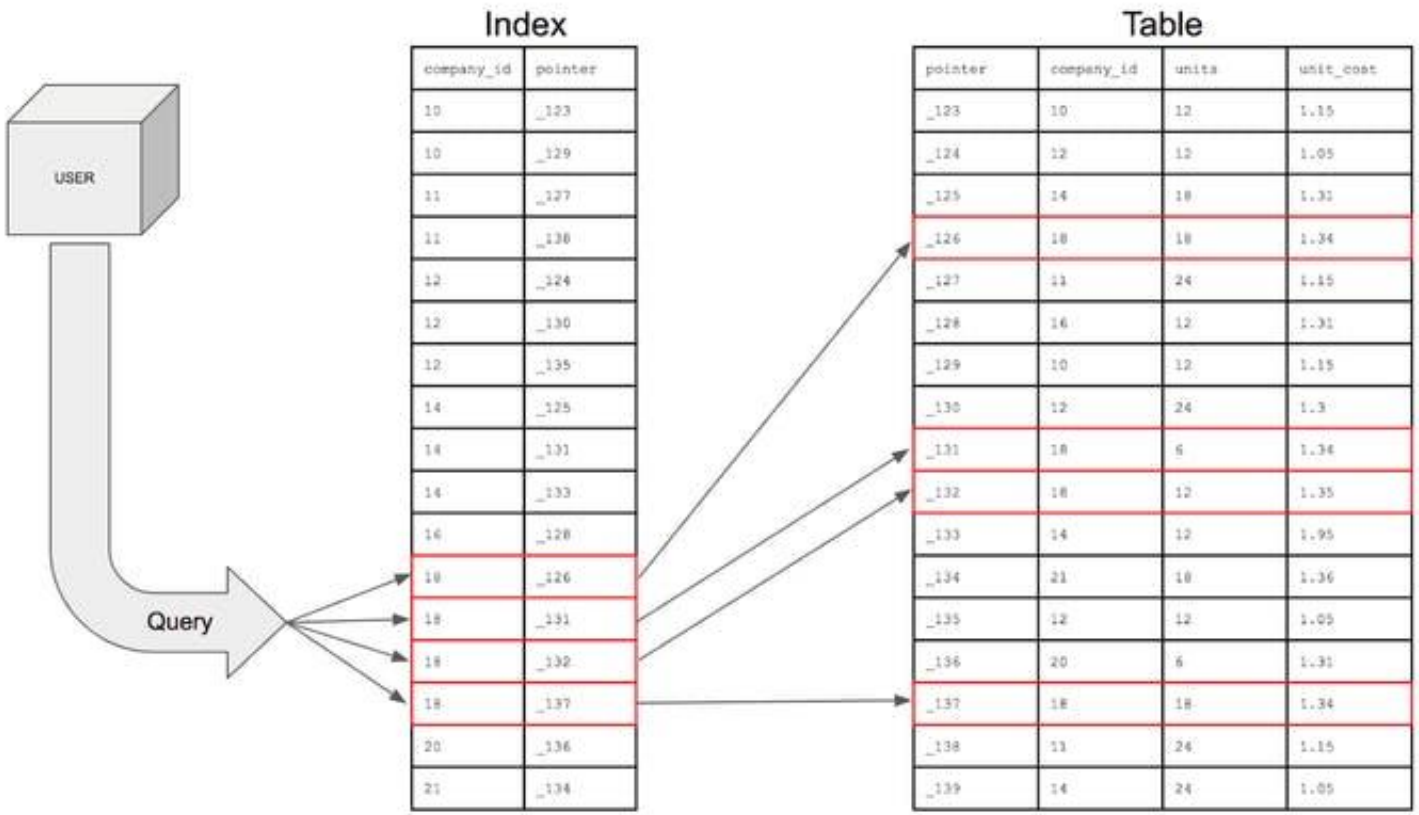
	A-Z country	123 sum
1	DE	600
2	NL	410
3	FR	300

Порядок

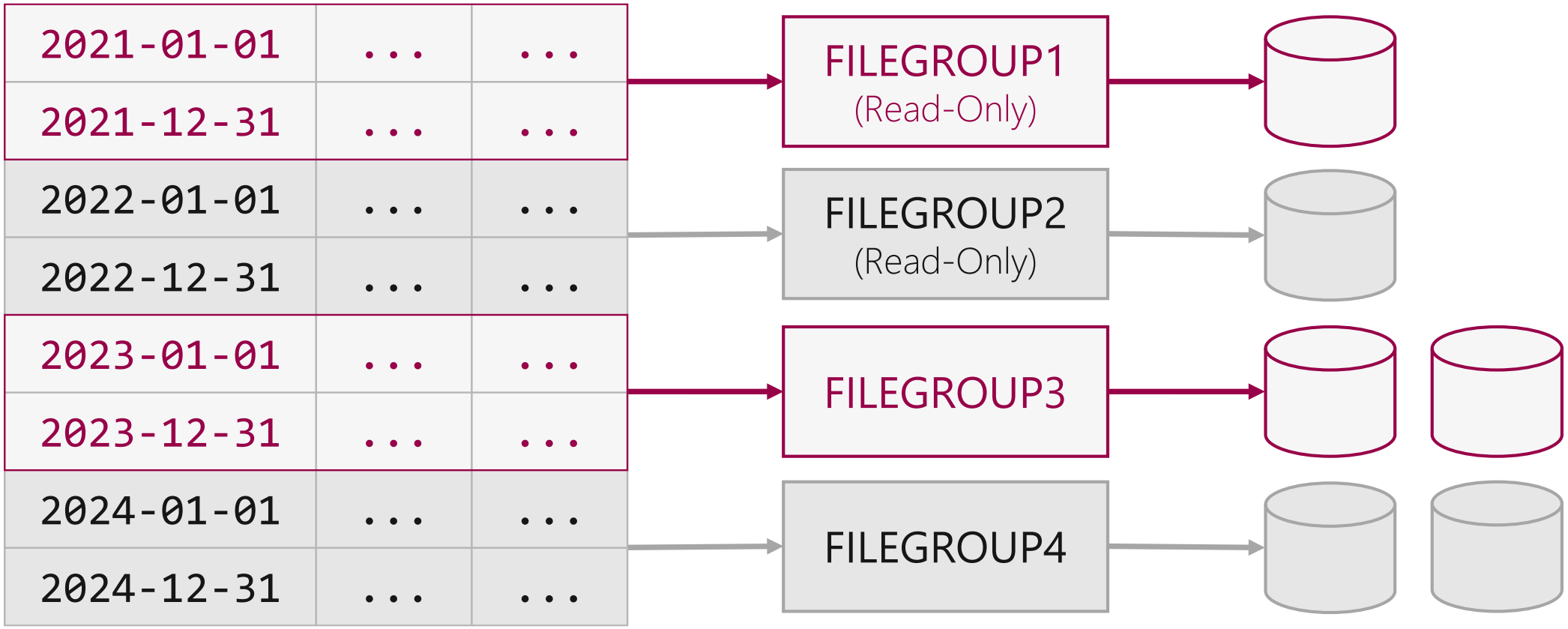


Индексы

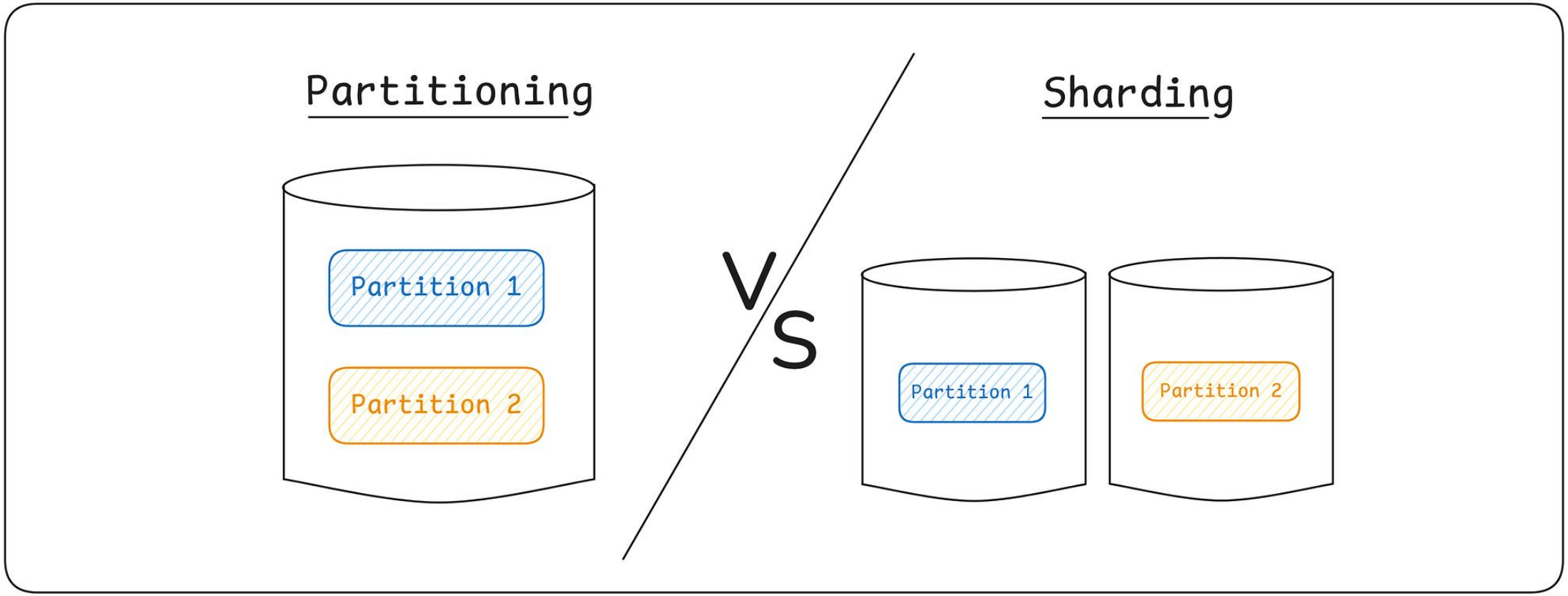
```
CREATE INDEX idx_user_id
    ON transactions(user_id);
```



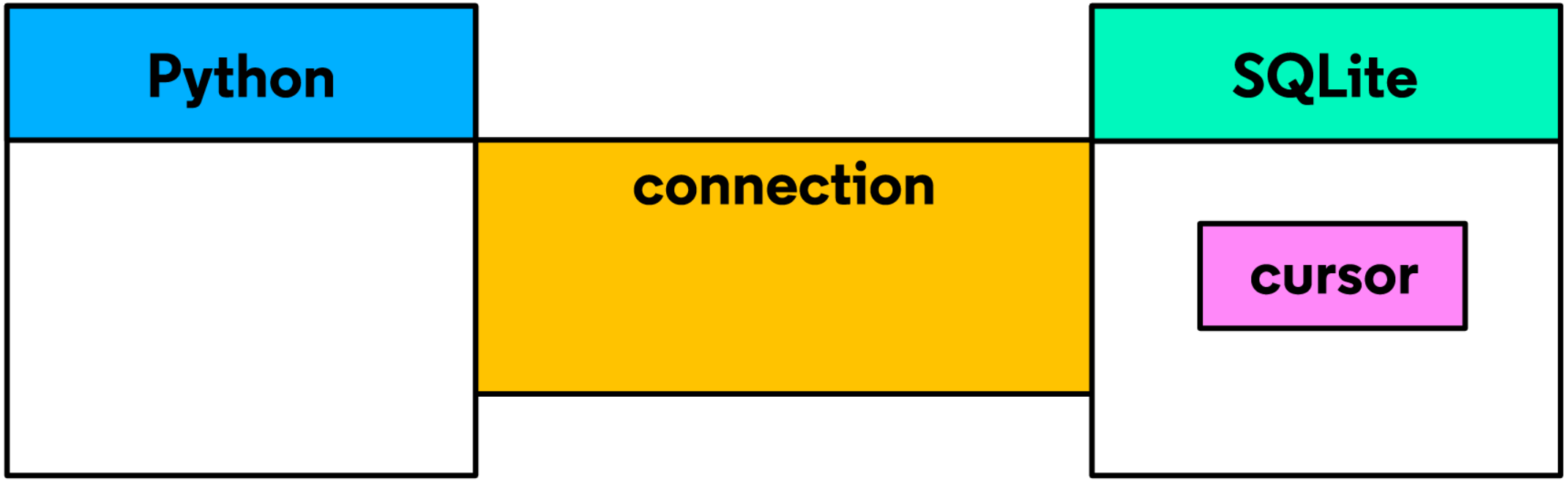
Партиционирование



Шардирование



Python + SQL



Python + SQL

```python
from sqlalchemy import create_engine

DATABASE_URL = "postgresql://postgres:1234@127.0.0.1:5432/postgres"
engine = create_engine(DATABASE_URL)
```

```python
with engine.connect() as conn:
    df = pd.read_sql("SELECT * FROM transactions", con=conn)

df
```

	transaction_id	user_id	amount	currency	event_time
0	1	1	100.0	EUR	2024-01-01 10:15:00
1	2	1	50.0	EUR	2024-01-02 11:00:00
2	3	2	200.0	EUR	2024-01-02 12:30:00
3	4	3	150.0	EUR	2024-01-03 09:45:00
4	5	3	-20.0	EUR	2024-01-04 13:20:00